

TDLWebshop

Épületgépészeti webshop és adminisztrációs rendszer

Szakdolgozat

Készítette:

Tóth Dávid László

Témavezető:

Dr. Bilicki Vilmos

Képzés / szak:

[ide írd be a pontos szakot]

Év:

2026

[Intézmény neve]

[Kar / tanszék neve]

Tartalmi összefoglaló

A szakdolgozat célja egy épületgépészeti termékekre specializált webshop és adminisztrációs rendszer megtervezése és megvalósítása. A TDLWebshop olyan webalkalmazásként készült, amely egyszerre támogatja a vásárlói folyamatokat, a termékkatalógus kezelését, a rendeléskezelést, a készletfigyelést, valamint az adminisztratív és dolgozói feladatokat. A rendszer Angular alapú kliensből, Firebase szolgáltatásokból, Firestore adatbázisból, biztonsági szabályokból és egy külön OpenRouter proxyval működő AI asszisztensből áll.

A dolgozat bemutatja a probléma hátterét, az MVP határát, a követelményeket, a felhasználói eseteket, az adatmodellt, a jogosultsági rendszert, a biztonsági megfontolásokat, valamint a tesztelési és validációs folyamatot. A fejlesztés során fontos szempont volt, hogy a rendszer ne csak funkciók halmaza legyen, hanem bírálható, reprodukálható és termékszerű szakdolgozati munka.

Abstract

The thesis presents the design and implementation of TDLWebshop, a web-based e-commerce and administration system focused on building engineering products. The application supports product browsing, cart management, checkout, user profiles, order tracking, administrative product and order management, stock monitoring, CSV import, invoice generation and an AI assistant connected through a server-side OpenRouter proxy.

The main emphasis of the thesis is not only the implementation itself, but also the engineering justification of the system: requirements, use cases, architecture, data model, access control, security, testing and reproducible deployment are documented as part of the final deliverable.

Tartalomjegyzék

A tartalomjegyzéket Wordben a véglegesítés előtt frissíteni kell: Hivatkozások -> Tartalomjegyzék -> Frissítés. Ez a dokumentum szöveges alap, a végleges oldalszámokat a képernyőképek és ábrák beillesztése után érdemes rögzíteni.

1. Bevezetés, problémafelvetés és célkitűzés

Az épületgépészeti termékek értékesítése sajátos terület, mert a vásárlók jelentős része nem egyszerűen terméket keres, hanem megoldást egy konkrét szerelési, fűtési, vízvezetékes vagy klímatechnikai feladatra. Egy általános webshopban gyakran nehéz áttekinteni, hogy egy termék milyen kategóriába tartozik, van-e készleten, hogyan rendelhető, és milyen adminisztratív folyamat kapcsolódik hozzá. A TDLWebshop ezt a problémát egy célzott, épületgépészeti fókuszú rendszerrel kezeli.

A fejlesztés célja egy olyan MVP megvalósítása volt, amely bemutatja a vásárlói és az adminisztratív oldal fő folyamatait. A vásárlói oldalon a termékböngészés, keresés, kosár, checkout, profil, kívánságlista és rendeléskövetés kapott hangsúlyt. Az admin oldalon a termékfeltöltés, CSV import, rendeléskezelés, helyszíni vásárlás, készletfigyelés, jogosultságkezelés és bizonylatgenerálás jelenti a rendszer szakdolgozati magját.

[KÉPERNYŐKÉP HELYE: Kezdőlap dark mode, kategória lenyílóval]

[KÉPERNYŐKÉP HELYE: Kezdőlap AI asszisztenssel, nyitott AI ablakkal]

1.1. MVP-határ

Az MVP határa tudatosan úgy lett meghúzva, hogy a rendszer működő webshopként és adminisztrációs felületként is bemutatható legyen, de ne vállaljon túl nagy külső integrációs kockázatot. A bankkártyás fizetés ebben az állapotban nem valódi fizetési gateway-en keresztül történik, hanem fizetési módként jelenik meg. A számla/bizonylat PDF formában generálódik, de nem helyettesít jogszabályi értelemben teljes körű számlázóprogramot.

- Vásárlói út: termékek böngészése, kosár, checkout, rendelésleadás, profil és státuszkövetés.
- Admin út: termékek kezelése, CSV import, készletfigyelés, rendeléskezelés, helyszíni vásárlás.
- Biztonság: Firebase Auth, Firestore szabályok, szerepkörök, tiltott felhasználók kezelése.
- Dokumentált korlát: a fizetés és az AI asszisztens csak MVP-szintű, kontrollált működésként szerepel.

2. Piaci és területi összehasonlítás

A piaci vizsgálat célja az volt, hogy a TDLWebshop ne elszigetelt gyakorlófeladatként, hanem valós webshop-folyamatokhoz viszonyítva legyen értelmezhető. A vizsgálat épületgépészeti és általános webshopok fő funkcióira koncentrált: terméklista, szűrés, kosár, checkout, profil, adminisztráció, készletkezelés, dokumentáció és ügyfélkommunikáció.

[ÁBRA HELYE: Piackutatási táblázat összefoglalója az Excel fájlok alapján]

A saját rendszer értéke elsősorban abban jelenik meg, hogy a vásárlói és admin folyamatokat egy szakdolgozati MVP-ben együtt kezeli. A helyszíni vásárlás, mentett vásárlók, dolgozói

jogosultságok, CSV import és AI asszisztens olyan kiegészítések, amelyek a rendszer termékszerűségét erősítik.

3. Követelmények és use case-ek

A követelmények meghatározásánál három fő szerepkör jelent meg: vendég vagy vásárló, dolgozó és admin. A szerepkörök közötti különbség nem csak a felületben, hanem a Firestore biztonsági szabályokban és az admin oldali jogosultságkezelésben is megjelenik.

Azonosító	Követelmény	Ellenőrzés
K-01	A vásárló tudjon terméket keresni, kosárba tenni és rendelést leadni.	Kézi checkout teszt
K-02	A rendszer validálja a hibás emailt és telefonszámot.	Checkout validációs teszt
K-03	Az admin tudjon terméket feltölteni és CSV-ből importálni.	Admin CSV import teszt
K-04	A dolgozó csak korlátozott admin funkciókat érjen el.	Jogosultsági teszt
K-05	A rendelés státuszváltása auditálható és készletváltozással járjon.	Order service teszt és kódrészlet
K-06	Az AI asszisztens ne találjon ki nem létező terméket.	AI asszisztens domain teszt

[ÁBRA HELYE: Use case diagram - vendég/vásárló, dolgozó, admin]

[ÁBRA HELYE: Checkout és rendelésleadás szekvencia diagram]

4. Felhasználói felület és UX

A felület kialakításánál cél volt, hogy a webshop modern, sötét tónusú, technikai hangulatú arculatot kapjon, miközben a fő vásárlói folyamatok áttekinthetők maradjanak. A dark/light megjelenés CSS változókkal kezelhető, így a layout nem változik meg témaváltáskor.

[KÉPERNYŐKÉP HELYE: Terméklista szűréssel/kereséssel: termékek oldal.]

[KÉPERNYŐKÉP HELYE: Termékadatlap: egy konkrét termék oldala.]

[KÉPERNYŐKÉP HELYE: Kosár: több termékkel.]

[KÉPERNYŐKÉP HELYE: Checkout validáció: hibás email/telefonszám példával.]

[KÉPERNYŐKÉP HELYE: Checkout sikeres rendelés: sikeres leadás után.]

[KÉPERNYŐKÉP HELYE: Profil/rendeléskövetés: vásárlói profil oldal.]

[KÉPERNYŐKÉP HELYE: Kívánságlista: kívánságlista oldal.]

5. Technológiai háttér

A kliensoldal Angular alapokra épül, mert komponensalapú szerkezete jól illeszkedik a webshop különálló nézeteihez: kezdőlap, terméklista, termékadatlap, kosár, checkout, profil és admin felület. A Firebase a hitelesítést, Firestore adatbázist, biztonsági szabályokat és hostingot adja. Az AI asszisztenshez külön Cloudflare Worker proxy készült, hogy az OpenRouter API kulcs ne kerüljön kliensoldali kódba.

Technológia	Szerepe	Döntés indoka
Angular	Frontend keretrendszer	Komponensalapú, jól strukturálható kliensoldal
Firebase Auth	Hitelesítés	Gyors MVP, Firebase ökoszisztéma
Firestore	Adattárolás	Valós idejű, dokumentumalapú webshop-adatok
Firestore Rules	Adatvédelmi és jogosultsági szabályok	Kliensoldali appnál kritikus védelmi réteg
Cloudflare Worker	OpenRouter proxy	API kulcs szerveroldali kezelése Spark Firebase mellett
GitHub Actions	CI ellenőrzés	Build és teszt reprodukálható bizonyítása

6. Architektúra és adatmodell

A rendszer architektúrája kliensoldali Angular alkalmazásból, Firebase backend szolgáltatásokból és külön AI proxyból áll. Az üzleti logika egy része Angular service-ekben jelenik meg, míg az adatelérési és jogosultsági korlátokat a Firestore szabályok biztosítják.

[ÁBRA HELYE: Komponens-architektúra ábra - Angular, Firebase, Worker, OpenRouter]

[ÁBRA HELYE: Adatmodell diagram - Product, Cart, Order, OrderItem, UserProfile, Coupon, Invoice, SavedCustomer]

Entitás	Szerep	Fontos mezők
Product	Termékadat és készlet	name, sku, category, price, stock, images
Order	Rendelés fejadatái	customer, items, totals, status, paymentMethod
OrderItem	Rendelési tétel	productId, name, quantity, unitPrice
UserProfile	Felhasználói profil és szerepkör	uid, email, role, disabled
SavedCustomer	Mentett helyszíni vásárló	name, email, phone, company, taxNumber, disabled

Coupon	Kupon és kedvezmény	code, value, usageLimit, active
Invoice	PDF bizonylat adatai	invoiceNumber, orderId, totals, issueDate

7. Megvalósítás

A megvalósítás fejezet a legfontosabb működési egységeket mutatja be: checkout, rendeléskezelés, készletváltozás, számlagenerálás, admin felület, CSV import, jogosultságkezelés és AI asszisztens. A kódrészleteknél nem teljes fájlokat, hanem rövid, magyarázható részleteket érdemes bemutatni.

[KÓDRÉSZLET HELYE: src/pages/checkout/checkout.ts 367-555: rendelés véglegesítése és validáció.]

[KÓDRÉSZLET HELYE: src/app/services/order.service.ts 41-127: státusz, audit és készlet tranzakció.]

[KÓDRÉSZLET HELYE: src/app/services/order.service.ts 222-267: helyszíni rendelés tranzakció.]

[KÓDRÉSZLET HELYE: src/app/services/order.service.ts 269-302: számlaszám generálás.]

[KÓDRÉSZLET HELYE: src/app/services/invoice.service.ts 9-154: PDF számla felépítése.]

[KÓDRÉSZLET HELYE: src/pages/admin/admin.ts 607-748: admin/dolgozói jogosultságok.]

[KÓDRÉSZLET HELYE: src/pages/admin/admin.ts 1181-1257: CSV import validáció és mentés.]

[KÓDRÉSZLET HELYE: src/app/services/chatbot-llm.service.ts 26-90 és 211-236: AI asszisztens domain- és katalóguslogika.]

[KÓDRÉSZLET HELYE: workers/openrouter-proxy/src/index.js 82-197: OpenRouter proxy és szerveroldali kulcskezelés.]

[KÉPERNYŐKÉP HELYE: Admin áttekintés: admin főnézet.]

[KÉPERNYŐKÉP HELYE: Admin termékkezelés/CSV import: admin termékek fül.]

[KÉPERNYŐKÉP HELYE: Admin készletfigyelés: készlet fül.]

[KÉPERNYŐKÉP HELYE: Helyszíni vásárlás: mentett vásárló kiválasztással.]

[KÉPERNYŐKÉP HELYE: PDF számla/bizonylat: generált PDF.]

[KÉPERNYŐKÉP HELYE: Admin felhasználó/jogosultság kezelés: admin felhasználók fül.]

8. Biztonság és jogosultságkezelés

A biztonsági minimum a rendszer egyik legfontosabb része, mert a webshop személyes adatokat, rendeléseket, szerepköröket és adminisztratív műveleteket kezel. A Firebase kliensoldali konfigurációja webalkalmazásnál önmagában nem titok, viszont a Firestore szabályoknak és a szerepköröknek szigorúan kell védeniük az adatokat.

[KÓDRÉSZLET HELYE: firestore.rules 25-76 - aktív felhasználó, admin és dolgozó jogosultság]

[KÓDRÉSZLET HELYE: firestore.rules 294-361 - products, orders, users, savedCustomers és audit szabályok]

- Admin és dolgozói szerepkörök külön kezelése.
- Tiltott felhasználó írási jogosultságának megvonása.
- Audit napló törlésének és módosításának tiltása.
- OpenRouter API kulcs szerveroldali Worker secretként kezelve.
- .env és valódi tokenek kizárása a repóból.
- Kuponok és rendelési adatok jogosultsági védelme.

9. Tesztelés és validáció

A tesztelés célja annak bizonyítása, hogy a kritikus vásárlói és adminisztratív folyamatok működnek. Az automata build és teszt mellett kézi tesztjegyzőkönyv is készüljön, mert a GUI/UX és jogosultsági folyamatok egy része így ellenőrizhető a legjobban.

Teszt	Elvárt eredmény	Bizonyíték
Regisztráció és bejelentkezés	A felhasználó belép, tiltott fiók elutasítva	Kézi teszt
Termékkeresés és kategória	A terméklista szűrhető és kereshető	Képernyőkép
Kosár	Mennyiség módosítható, tétel törölhető	Kézi teszt
Checkout validáció	Hibás email/telefon nem enged tovább	Képernyőkép
Rendelés létrehozás	Rendelés létrejön, összegzés helyes	Kézi teszt
Admin státuszváltás	Státusz és készlet konzisztensen változik	Kézi/kódteszt
CSV import	Valid sorok menthetők, hibás sorok jelöltek	Admin képernyőkép
AI asszisztens	Csak domain témában válaszol, nem talál ki terméket	Kézi teszt

[KÉPERNYŐKÉP HELYE: GitHub Actions zöld CI futás]

10. Reprodukálhatóság, CI és üzemeltetés

A repó akkor bírálható nyugodtan, ha tiszta környezetben is elindítható. Ehhez README, .env.example, telepítési lépések, demo szerepkörök, CI futás és deploy leírás szükséges. A node_modules, build mappák, lokális cache-ek és valódi secret értékek nem részei a beadandó forráskódnak.

1. Függőségek telepítése: npm install.
2. Lokális indítás: npm run start.
3. Build ellenőrzés: npm run build.
4. Automata tesztek: npm test -- --watch=false.
5. Firebase hosting deploy: firebase deploy --only hosting.
6. OpenRouter Worker deploy: workers/openrouter-proxy mappából npx wrangler deploy.

11. Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat készítése során mesterséges intelligenciát támogató eszközöket is használtam. Ezeket nem önálló fejlesztőként kezeltem, hanem asszisztensként: ötletelésre, hibakeresési irányok keresésére, kódrészletek ellenőrzésére, dokumentációs vázlatok rendezésére és tesztelési szempontok összegyűjtésére. A végső döntések, a kód átnézése, a futtatás és a validáció az én feladatom maradt.

Az AI használata leginkább akkor volt hasznos, amikor több lehetséges megoldás közül kellett választani, például a Firebase Spark csomag korlátai miatt az OpenRouter API kulcsot nem Firebase Function secretként, hanem Cloudflare Worker oldalon kellett kezelni. Ilyenkor az AI segített alternatívákat megfogalmazni, de a választott megoldást a projekt korlátai, a biztonsági szempontok és a futtathatóság alapján ellenőriztem.

A fejlesztés során előfordult, hogy egy AI által javasolt megoldás túl általános vagy a projekt szempontjából nem megfelelő volt. Ilyen példa volt az AI asszisztens termékajánlási logikája: a cél nem az volt, hogy a modell tetszőleges terméket kitaláljon, hanem hogy a saját katalógus alapján, ellenőrzött módon ajánljon. Ezért a kliensoldali logikában külön szűrés készült, amely csak releváns katalógustalálát esetén jelenít meg terméket, más esetben pedig szakmai irányt és kapcsolatfelvételi javaslatot ad.

Tudatosan nem használtam AI-t a dolgozat személyes következtetéseinek végleges megfogalmazására és a projektértékelés felelősségének átvállalására. Ezeknél a részeknél fontos, hogy saját tapasztalat alapján jelenjen meg, mit sikerült megvalósítani, milyen korlátok maradtak, és mit fejlesztenék tovább. Az AI által adott szövegeket ezért át kell írni saját nyelvezetre, és csak ellenőrzött, megértett tartalom kerülhet a végleges dolgozatba.

A rendszer maga is tartalmaz AI funkciót: a vásárlói AI asszisztens OpenRouter modellen keresztül működik, de a hívás nem közvetlenül a böngészőből történik. A kliens egy Cloudflare Worker proxyt hív, ahol az API kulcs secretként van kezelve. A megoldás korlátja, hogy a modell

válaszai nem tekinthetők hivatalos műszaki tanácsadásnak, ezért a felületnek óvatosan kell kommunikálnia: pontos termékajánlat vagy beszerezhetőség esetén email-es vagy személyes egyeztetés javasolt.

12. Összefoglalás és továbbfejlesztési lehetőségek

A TDLWebshop szakdolgozati MVP-ként bemutatja, hogyan lehet egy épületgépészeti webshopot vásárlói és adminisztratív oldalról is felépíteni. A rendszer erőssége, hogy a termékböngészés, kosár, checkout, profil, admin rendeléskezelés, CSV import, készletfigyelés, jogosultságkezelés, PDF bizonylat és AI asszisztens egy közös folyamatba illeszkedik.

Továbbfejlesztési irányként valódi online fizetési integráció, hivatalos számlázó rendszerhez kapcsolódás, részletesebb készlet-előrejelzés, fejlettebb B2B árképzés, valamint szerveroldali üzleti logika bővítése jelölhető meg. Ezek nem az MVP hiányosságai, hanem olyan irányok, amelyek egy éles kereskedelmi rendszer felé vinnék tovább a projektet.

[SAJÁT ZÁRÓGONDOLAT HELYE: ide írd 8-12 mondatot arról, mit tanultál a projektből, melyik része volt a legnehezebb, és mit csinálnál másként egy következő verzióban.]